USD8 security audit

Mainnet readiness review

15 October 2026 | Current working tree | Prepared with Codex

Recommendation: hold a large-TVL launch until the governance, TEE provisioning and verification gates in this report are closed. This review did not reproduce a permissionless Critical or High-severity funds-drain in the reviewed configuration.

The review covers the latest USD8 contracts in the supplied folder, including uncommitted deployment changes. It combines source review, the existing test suite, 110,000 selected fuzz cases, Rust/Solidity integration tests, static analysis, custom adversarial reproductions and a pinned Ethereum mainnet-fork deployment exercise.

Finding	Severity / prerequisite	Observed consequence
M-01: no individual recovery from an unusable standing root	Medium, conditional on a faulty or malicious authorized root	Escrow and bonds remain inaccessible after expiry until governance intervenes.
L-01: residual rewards can move between LP cohorts	Low; extreme share inflation after repeated large losses	A completed 70 USD8 stream credits zero to the old LP; a later entrant receives it.
L-02: fee recipient can break claim execution	Low; privileged configuration error	A pool selected as fee receiver makes otherwise valid payouts revert.

The Medium and fee-recipient findings are fault-containment weaknesses at trusted boundaries, not unauthorized privilege escalations. The reward finding is reproducible but requires an extreme economic state; the reproduction uses a governance-selected 99.99% incident cap, not the 50% launch default.

Evidence in brief

Default suite: 537 passed, 0 failed, 5 skipped. All five skipped settlement integration tests then passed with a freshly rebuilt Rust binary. Eleven existing fuzz tests passed 10,000 cases each. Three new adversarial tests reproduced the concerns, and one new mainnet-fork test passed. The separate formal-property profile failed to compile.

No production source was patched, no transaction was broadcast, and no deployment or account setting was changed. The attached evidence contains test code, logs, a source snapshot, file hashes and reproduction instructions.

1. Scope and version boundary

Repository: /Users/rickchen/codes/usd8Codes/usd8core

HEAD: 6885131ef0c88656b5d519c97c4b8229348f3d78. This is not a clean-commit-only audit: the inspected working tree contains changes beyond HEAD, including the revised two-stage deployment and the explicit bootstrap owner constructor.

Primary source	Review emphasis
USD8.sol; SharedBase.sol	Mint/burn authorization, initialization, upgrades, shared access controls and protected sweeping.
Treasury.sol; strategies/StrategyBase.sol; strategies/ERC4626Strategy.sol	USDC backing, distressed redemptions, strategy accounting, rounding and revenue routing.
Registry.sol	Roles, canonical addresses, pool topology, incident freezes, emergency clearing and policy changes.
DefiInsurance.sol	Incident opening, signed roots, claim-set commitments, eligibility proofs, budgets, escrow and recovery.
SingleAssetCoverPool.sol	ERC-4626 entry, asynchronous exits, reserved withdrawals, rewards, loss accounting and sweeping.
oracles/USD8PriceOracle.sol	Composite valuation, reserve ratio, round metadata and historical-query limitations.
adapters/USD8SavingsAdapter.sol; deployment/USD8SavingsBootstrap.sol	Morpho accounting integration, reward checkpointing, seed and ownership setup.
Two interfaces and production deployment scripts	Integration contracts; CREATE3 salts, atomic initialization and governance hand-off.

There are 13 Solidity files under src/, including nine deployable contracts, two abstract bases and two interfaces. The code under test was frozen into /tmp/usd8-audit-20261015. Production source and deployment file hashes still matched that snapshot when the evidence was packaged.

Launch scope versus later activation

The current deployment installs USD8, Treasury, Registry, the wstETH cover pool, DefiInsurance and the composite oracle. It launches with no yield strategy and no savings vault. Strategies and sUSD8 were reviewed as shipped code, but their future live addresses, parameters and external risk remain separate activation gates.

Supporting material

Reviewed relevant tests, deployment configuration, the earlier BONISTECH report and fix report, decisionTradeOffs.md, and selected Rust settlement interfaces. The complete Rust service, Nitro enclave, infrastructure, frontend, booster implementation and all vendor contracts were not independently audited end to end.

2. Assets and trust model

Asset / boundary	Protection and remaining trust
Treasury USDC	Minting issues 1e12 USD8 units per USDC base unit. Healthy redemption is capped at peg; distressed redemption uses the measured backing ratio. Approved strategy values are trusted.
Cover-pool principal	Internally accounted assets resist donation-based share-price inflation. Matured exits move into a separate withdrawal reserve. Claim payouts reduce active principal only.
USD8 LP rewards	rewardReserve protects distributed rewards from sweeps; a fixed-point accumulator assigns them to earning shares. Exit-escrow shares stop earning immediately.
Claim tokens, bonds and boosters	Claims hold assets until cancellation, recovery or proof-backed resolution. Correct eligibility and allocations depend on authorized off-chain computation.
Timelock / beacon	Controls upgrades and sensitive policy. Beacon ownership is separate from Registry.timelock and is not disabled by endBetaMode.
Operational admins	Can pause, route Treasury profit and allocate approved strategies. During beta they can replace or void a standing root. These are substantial trusted powers.
TEE signers	One authorized signer suffices. The contract binds the signed PCR hash but does not independently verify a Nitro attestation or recompute economics.

What was treated as an attacker capability

Unprivileged users may transact, donate, transfer shares, request exits, join/cancel claims, relay signatures, supply Merkle proofs and choose transaction order. Approved external assets are assumed to obey their documented transfer/accounting model. Privileged mistakes and signer failures were tested separately and are explicitly identified.

What an on-chain signature proves

Settlement signatures bind chain/domain, contract, incident, root, unresolved-claim count, ordered pools, payout budgets, claim-set commitment and the incident PCR commitment. Those bindings resist replay and substitution. They do not prove that every claim is included in the root, that historical score is correct, or that settlement data will remain available.

Limits on loss are policy, not a fixed 50% guarantee

The launch cap is 50% of each pool per incident, but governance may set any positive value below 100%. Multiple authorized incidents can consume capital repeatedly. Admin or signer compromise therefore has material financial consequences even when all local budget checks work as designed.

3. M-01 - Unusable roots block individual recovery

Severity: Medium, conditional. Confidence: high for the demonstrated behavior. Requires an erroneous/malicious authorized settlement or beta correction, or unavailable proof data that cannot be independently reconstructed. No permissionless way to install such a root was found.

Affected code

src/DefiInsurance.sol:653-688, 695-717 and 754-845; specifically the proofless branch at 771-780 and mandatory proof check at 793-799. Registry.clearDefiInsurance at src/Registry.sol:524-535 is the available global recovery mechanism.

Root cause

A nonzero standing root permanently selects the proof-backed resolution path while this module remains registered. Expiry only disables accepting a payout; it does not open an individual proofless refund. The root submission checks signature and pool budgets, but cannot check that its leaves actually cover the committed claim set.

Reproduction

- A claimant escrows 100 insured tokens and a 10 USD8 bond in an otherwise ordinary incident.

- An authorized signer supplies a nonzero root with no valid leaf for that claimant. The contract accepts the signature and zero payout budget.

- After advancing a year, the incident is inactive but finalizeClaim(..., false, ...) still reverts InvalidProof. The 100 tokens and 10 USD8 remain in escrow.

- The timelock clears the registered insurance module; the same claimant can then use the existing proofless recovery branch and receives both balances.

Evidence: test_Audit_MalformedRootLocksEscrowBeyondExpiryUntilModuleCleared in tests/AuditAdversarial.t.sol; adversarial.log. The signer is intentionally modeled as faulty in this test. This does not demonstrate a defect in the honest Rust root generator.

Impact and qualification

An affected claimant cannot independently recover after the payout opportunity has ended. The lock can be indefinite without governance action, but is not unconditionally permanent: clearing the module works. A global clear disrupts normal insurance and recovery must occur while the module remains inactive. Correct, publicly reconstructible roots do not trigger this issue.

Recommended remediation

Add a bounded post-expiry, per-incident recovery path that cannot pay insurance or spend score. Preserve bond deterrence: for example, allow proofless return of insured tokens/boosters while forfeiting the bond, and retain proof-backed eligible bond refunds. Alternatively add a tightly scoped timelock void for an expired incident that enables refunds without clearing the live module. Specify the policy before implementation.

Acceptance test

After an omitted or malformed leaf and expiry, an individual can recover the appropriate escrow without changing global topology. The path must not let an ineligible claimant bypass the bond penalty before the agreed timeout, reopen payouts, resolve twice, or alter an active later incident.

4. L-01 - Rewards can migrate after share inflation

Severity: Low. Confidence: high. This is a residual limitation of the earlier reward-rounding fix. It requires earning share supply to exceed the entire scaled reward stream; it was not reproduced in the normal launch state.

Affected code

src/SingleAssetCoverPool.sol:462-485 and 549-598. rewardResidual is carried globally, while user entitlement is updated only from rewardPerShareStored.

Root cause

The carry fixes loss from fragmenting a stream into many checkpoints, but does not guarantee a representable increment over the full reachable share-supply range. If earningShares > emittedReward * 1e30, the accumulator increment remains zero for the whole stream. That residual has no per-user ownership. When old holders exit, later earning shares can receive it.

Concrete observed state

The test starts with a 1,000 USDC deposit, sets the permitted incident cap to 9,999 bps, sets claim fees to zero and ends beta. Ten signed, fully finalized loss-and-replenishment cycles create approximately 9.9992e51 share base units using ordinary contract calls. No storage patch or artificial minting of cover shares is used. Mock tokens supply the test principal, and signed loss economics are assumed for these cycles.

A 70 USD8 reward over seven days has only about 7e49 scaled emission units. The old LP receives zero. After its complete exit, a new entrant deposits 1,000 USDC; a new 1 USD8 stream and one second of elapsed time let that entrant claim 70.000001653438705439 USD8, including the old cohort's residual. The old LP still has zero earned rewards.

Impact and prerequisites

Historical LP rewards can be deferred for an unbounded period and reassigned to later participants. This is reward entitlement loss, not a demonstrated theft of principal. The cohort-reset reproduction uses an unseeded pool, which the public initializer permits. The production script uses an irrecoverable seed, so the exact full-reset payout is not established for that seeded pool. The representability threshold itself remains relevant whenever share inflation is sufficiently large.

At the 50% default cap this requires far more loss/replenishment cycles than in the test. It must not be described as a practical launch-day permissionless attack. Governance still permits the much faster 99.99% configuration used by the reproduction.

Recommended remediation and acceptance test

Constrain supported share supply relative to the reward precision, or redesign reward accounting so undistributed scaled value remains attributable to the earning cohort across exits. Merely increasing the constant or carrying one global remainder does not solve the full-range problem. Define a migration/reset policy before reaching unsafe share prices; retaining the default loss cap is a useful mitigation, not a proof.

Test both seeded and unseeded pools through repeated losses and replenishment. Past earned rewards must remain recoverable after exit, and newcomers must not receive a material previous-cohort residual. Evidence: test_Audit_ResidualFixStillLosesWholeStreamAfterReachableShareInflation; adversarial.log.

5. L-02 - Fee recipient can disable valid payouts

Severity: Low, privileged misconfiguration. Confidence: high. This does not give an attacker a new admin capability; admins can already pause payouts. It is a preventable configuration and liveness hazard.

Affected code

src/Registry.sol:273-283 accepts any nonzero protocol fee receiver, including a cover pool. src/DefiInsurance.sol:891-907 reads the current receiver during payout. src/SingleAssetCoverPool.sol:501-510 rejects payClaim to the pool itself.

Root cause

The incident snapshots its fee percentage but not its destination. The recipient may change during an incident, and the setter checks neither self-payment incompatibility nor payout-token transfer restrictions. Fee payment is atomic with the claimant payment, so a recipient that cannot receive one row rolls back that claimant's entire finalization.

Reproduction

- An LP provides 1,000 USDC. A claimant registers, and a correct leaf promises 100 USDC with a 125 USDC gross pool budget at the default 20% protocol share.
- After the correction phase, the admin changes the fee receiver to the paying cover pool.
- The claimant submits the correct proof. Its payment is rolled back when the protocol-fee transfer calls payClaim on that pool with the pool itself as recipient.
- Restoring a valid recipient makes the identical claim succeed and pays the claimant 100 USDC.

Evidence: test_Audit_FeeReceiverCanDisableOtherwiseValidClaimDuringFinalization in tests/AuditAdversarial.t.sol; adversarial.log.

Impact

Claims involving the incompatible pool cannot be accepted while the setting remains. If this spans the fixed finalization window, the claimant loses the insurance payout opportunity, although a valid proof can still support a decline and escrow return. A transfer-restricted token or blacklisted recipient can create a similar external dependency, but the concrete test uses the pool's own explicit recipient rejection.

Recommended remediation

Reject registered cover-pool recipients where self-payment can occur, and choose whether fee destinations should be snapshotted per incident. Snapshotting alone does not fix an already incompatible recipient. For stronger liveness, separate fee accrual from delivery so a fee transfer failure cannot block the claimant, while carefully preserving pool budget and reserve accounting.

Acceptance test

A valid fee update remains possible, but no accepted configuration can introduce the demonstrated self-payment failure. If a fee destination becomes unusable externally, the claimant can still complete the approved payout without exceeding the signed gross budget. Test multi-pool rows and token restrictions explicitly.

6. Validation results and reproducibility limits

Check	Observed result	Interpretation
Default Foundry suite	537 passed; 0 failed; 5 skipped; 36 suites	Includes existing stateful invariants. A green suite is evidence for tested domains, not universal correctness.
Rust / Solidity integration	5 passed; freshly rebuilt usd8-ffi	Claim-set replay, EIP-712 digest, Merkle payouts, boosters and underfunded multi-claimant settlement matched.
Expanded selected fuzzing	11 tests x 10,000 cases; all passed	110,000 cases for accounting, ratio, booster, settlement boundaries and savings behavior. Existing tests, not 11 additional distinct checks.
New adversarial tests	3 passed by asserting the concerning behavior	Reproduce M-01, L-01 and L-02. Passing these is not a statement that the issues are fixed.
Pinned mainnet-fork exercise	1 passed at block 26,196,372	Deployed current system locally against real dependency code; exercised USDC mint/redeem and hand-off.
Slither 0.11.5	70 contracts; 101 detectors; 124 raw alerts	Manually triaged by reachable code path and trust model. Raw alerts are not 124 confirmed vulnerabilities.
Formal-property profile	Compilation failed	Calls removed ERC4626Strategy.swap. No current profile pass and no solver proof claimed.

Toolchain

Forge 1.5.1-stable (b0a9dd9ceda36f63e2326ce530c10e6916f4b8a2); compiled Solidity metadata reports 0.8.28+commit.7893614a, optimizer enabled, runs=1, EVM Prague. The separate kontrol profile explicitly requests Cancun. Pin the actual production profile; these are distinct build configurations.

Default invariant settings resolved to 256 runs, depth 500 and fail_on_revert=false; individual test annotations may override settings. This review did not run the separate deep profile or a Kontrol solver campaign. It did intensify the 11 listed stateless fuzz tests to 10,000 cases each.

Resolved environment issue

The default rustc 1.91.1 could not build packages requiring 1.92. The already installed 1.94.1 toolchain rebuilt the current source successfully, after which all five FFI tests ran. The old prebuilt binary was not used as current-source evidence.

7. Release gates requiring action

R-01 - Verify governance before user deposits

The mainnet configured admin, 0xB2E999D531D45a9115dA7706adFc651999f3c1F1, had zero deployed code at the pinned block. It is the sole configured proposer/canceller and an immediate operational admin. No on-chain multisig protection was established. An EOA may use off-chain custody controls, but those controls were not supplied or verified.

The first deployment step leaves the broadcaster as Registry timelock and pool-beacon owner. The second step transfers both to a 24-hour timelock. Keep user funds out until both steps and role read-backs are complete. Set a reviewed Safe or otherwise document and validate the intended custody controls before substantial TVL. The source already warns that privileged beta powers are trusted.

R-02 - Provision and verify the actual TEE trust chain

The current deployment scripts do not set a PCR hash or register TEE signers. The fork test confirms teePcrHash remains zero after deployment. User-driven signed openings are unavailable until signer provisioning; the Rust opening path also rejects a zero PCR commitment. Admin fallback opening does not replace a working settlement signer.

Before launch, record the enclave build/PCR, verify the signer-to-attestation binding, set the correct on-chain commitments and signer addresses, and execute an end-to-end incident with the actual production runtime and archive-RPC path. Reproduction of Merkle/ABI computation alone does not attest the production enclave.

R-03 - Reconcile the verification package to the release

FOUNDRY_PROFILE=kontrol forge test fails at formal-verification/properties/ERC4626StrategyAdversarialHarness.k.sol:525 because strategy.swap no longer exists. ERC4626StrategyHarness.k.sol:71 also references it. USD8SavingsBootstrapHarness.k.sol:398 and 436 instantiate the now-argument-taking bootstrap with no arguments.

Update the harnesses and reviewed inventory, rebuild and run the current executable properties, and refresh their source/build hashes. The formal documents themselves report zero accepted solver proofs. Their older Forge-green counts must not be presented as validation of this working-tree release.

Priority order

- Before large TVL: complete R-01/R-02, close the M-01 recovery-policy decision, and restore the intended release verification gate.

- Before deployment freeze: address L-02 and explicitly remediate or accept L-01 with bounded supported parameters and monitoring.

- Before enabling strategies or sUSD8: independently approve their deployed dependencies and configuration, then repeat integration and failure-path testing.

8. Mainnet dependency and deployment checks

Read-only chain checks and the local fork were pinned to Ethereum mainnet block 26,196,372, hash 0x6807558a7267e7ef6137c36e189564a3687eaa69facf74d2090531d8d1da418c. Block timestamp: 15 October 2026, 09:18:47 Europe/Berlin (07:18:47 UTC). One public RPC provider supplied these reads; no independent state proof or second-provider quorum was collected.

Configured dependency	Observed at the pinned block
USDC: 0xA0b8...6eB48	Real token code supported a locally funded 1,000 USDC mint and full redemption through the newly deployed Treasury.
wstETH: 0x7f39...E2Ca0	Real token code accepted the local 0.001 wstETH pool seed through current deployment logic.
Cover oracle: 0x164b...8bCe0	description() = wstETH / USD Calculated. Positive answer; updated 6,384 seconds before the block.
USDC oracle: 0x8fFf...818f6	description() = USDC / USD. Positive answer; updated 77,892 seconds before the block. Both feeds were within the 36-hour default bound.
Booster: 0x6f74...6f12d	9,168 bytes of code and ERC-1155 interface support. This is not a review of its mint policy, burn permissions or upgrade/admin design.
Configured admin: 0xB2E9...3c1F1	No deployed code. Contract-based multisig protection not verified.

What the fork exercise establishes

The current CREATE3 deployment code can use the existing factory and configured dependencies at this block. Constructor-time proxy initialization, pool seeding, Registry wiring, USD8 mint/redemption and the hand-off logic succeeded locally. Test funding cheats changed only the fork. There was no broadcast.

What remains untested operationally

The test calls script internals through a Solidity harness; it does not reproduce a hardware-wallet broadcast or interruption between separate deployment transactions. The hand-off precheck requires both governance handles still belong to the interim deployer. A partial live hand-off therefore needs a verified resume/manual-completion runbook.

Bytecode headroom

All nine first-party deployable runtime sizes were below 24,576 bytes in this build. DefiInsurance is closest: 23,695 bytes, leaving 881 bytes. A recovery fix can materially consume this headroom; rerun size checks and deployment simulation after changes. Full compiler settings and bytecode SHA-256 hashes are in build-manifest.json.

9. Prior audit follow-up and scanner triage

The repository includes a BONISTECH review of commit 6efab51299d551ed45b20d82b305d300e9f01d7b and a fix report referencing 460961d. This review checked those code paths in the present snapshot rather than inheriting the earlier report's conclusions.

Prior item	Current assessment
M-01: admin claimant sweep reentrancy	Original path is addressed: shared sweep functions use the same transient reentrancy guard, and cancellation/proofless refund retain insured-token escrow until transfer. Three sweep regression tests passed.
L-01: rewards lost to repeated checkpoint flooring	Original fragmentation scenario is addressed by rewardResidual; three regressions passed. Full-range entitlement preservation is incomplete under the new L-01 state demonstrated in this report.
L-02: virtual-share addition overflow	Mint/deposit headroom excludes the virtual-share offset. Existing headroom and partial-exit tests passed. This does not establish correctness for all unrelated reward arithmetic.
L-03: value loss on strategy deposit rounding	Treasury now measures aggregate reserves and rejects loss above 100 USDC base units. Two strategy-deployment regression tests passed. Approved vault behavior remains trusted.
L-04: stale scheduled module clear	clearDefiInsurance binds expected module and active incident. Existing suite passed; emergency handling for a reverting module is intentional.

Why raw scanner severity was not copied into findings

Slither labels the reward remainder modulo as weak randomness, although it is not used for randomness. SharedBase.sweepETH is intentionally access-controlled. USD8.transfer return values are known from the canonical token implementation. scoredRates is initialized through append operations. These alerts are not standalone vulnerabilities.

Many reentrancy reports conservatively model arbitrary external token/strategy hooks and do not capture the complete transient-guard/access-boundary behavior. Reviewed the guarded fund-moving paths and existing callback regressions. Loop alerts remain relevant to governance limits, approved pool/strategy count and bounded exit processing; they do not show a launch-configuration gas exploit by themselves.

The 124 raw detector outputs are preserved in slither.json and slither.log for independent follow-up. This report makes no claim that automated analysis establishes absence of defects in approved external contracts or future implementations.

10. Coverage of important security properties

Area	Reviewed property and evidence / limit
USD8 issuance and permits	Treasury-only mint/burn, disabled implementation initialization, one-time proxy init and beta-gated UUPS authorization. Existing tests passed. Future upgrades remain trusted.
Reserve integrity	Healthy surplus and distressed backing checks, minUsdcOut, rounding and reserve-preserving strategy deposits. Peg fuzz tests passed at 10,000 cases. Generic six-decimal assets are not equivalent to USDC.
Strategy failures	Value reads fail closed; failed withdrawals are skipped; partial external liquidity can still block a large redemption. These are documented design dependencies, not newly proven theft paths.
Revenue delivery	Weighted splits, final-receiver remainder, exact hook pull and allowance reset. Admins can redirect profit; receiver failure reverts distribution.
Pool principal and exits	Donation immunity, virtual shares, supply headroom, escrowed exits, FIFO timing, separate withdrawal reserve and incident-open settlement of matured exits. Existing stateful tests passed.
Pool rewards	Transfer checkpoints, reward reserves, zero-earning-share deferral and callback protection. Fixed-point ownership limitation reproduced in L-01.
Insurance authentication	Typed-data domain, incident/pool/claim-set commitments, root replacement restrictions and booster-consistent leaves. Fresh Rust integration checks passed. Economic truth remains off-chain.
Claim settlement and refunds	Escrow/bond accounting, payout caps, decline paths, exact phase boundaries, module removal and stale topology. M-01 and L-02 identify recovery/configuration exceptions.
Oracle	Backing-ratio agreement fuzzed against Treasury. Historical rounds intentionally limited; timestamps are inherited from the USDC feed. Freshness must be checked by consumers, including off-chain settlement.
Savings integration	Parent-vault-only adapter hooks, signed allocation deltas, pre-profit checkpoint, ownership/allocator hand-off and seed mechanics. Existing tests/fuzz passed. No production savings vault is launched by current scripts.
Governance / deployment	CREATE3 guards, constructor-time initialization, mainnet address semantics, two governance handles and timelock role hand-off. Local fork passed; real custody and interrupted broadcasts remain release tasks.

Coverage means the code path was examined and relevant concrete tests were run. It does not mean each property is formally proved, exhaustively fuzzed, or valid for arbitrary external tokens, strategies, future upgrades or governance configurations.

11. Remediation and release acceptance

Required evidence before revisiting the launch recommendation

- Freeze a clean release commit and attach exact source, dependency and compiler settings. Include deployment changes currently outside HEAD.

- Select an explicit M-01 refund/bond policy and prove that expired malformed-root claims recover without disrupting a later incident. Recheck double resolution and score/budget isolation.

- Prevent L-02 self-payment failures and test fee-recipient behavior through the entire finalization window, including multiple pools.

- Define supported pool share/reward ranges for L-01. Exercise large losses and cohort transitions, including the actual dead-seeded deployment configuration.

- Repair and run the formal-property profile against the release. Keep executable test results separate from solver-proof claims.

- Use verified production governance addresses; confirm proposer/canceller roles, open execution, 24-hour minimum delay, no interim deployer admin if distinct, Registry.timelock and beacon.owner.

- Provision the real PCR and signer set, publish independently reproducible settlement inputs/artifacts, and test archive-RPC availability and emergency signer/module recovery.

- Repeat mainnet-fork lifecycle tests after the fixes and verify runtime size, storage layout for any upgrade, dependency addresses and launch caps.

Accepted risks needing explicit ownership

The timelock can change strategies and policy; beta permits upgrades and immediate admin root correction. The 24-hour governance delay is shorter than the 7-10 day pool exit process, so an LP cannot necessarily exit before a queued malicious upgrade. Ending beta does not renounce the independent pool beacon. These are trust assumptions to disclose and govern, not barriers enforced by the tested code.

Reserve strategies can be illiquid or misreport/revert; USDC can restrict transfers; oracle values can be stale or external feeds unavailable; insured tokens need compatible accounting and historical replay. A signer or approved enclave/RPC failure can stop honest payouts. Treasury and insurance capital are separate, but common governance and oracle dependencies remain.

Suggested operational checks

Alert on upgrades, role changes, module clearing, signer/PCR changes, fee recipient changes, large reward residuals, abnormal share-to-asset ratios and failed claims. Keep a rehearsed global-clear recovery procedure until individual recovery is improved. Configure each future strategy and savings vault as its own reviewed release, not merely as an address added to a list.

No fixes are included in this deliverable. The evidence tests intentionally demonstrate current behavior; after remediation, convert them into assertions of the intended safe behavior and rerun the full relevant suite.

12. How to reproduce the evidence

Use the exact working-tree snapshot identified by source-manifest.json and its pinned dependencies. Copy the two evidence/tests files into the repository test/ directory on a disposable copy. They import existing project test fixtures. The source snapshot contains production Solidity and deployment files; vendor sources and the whole test tree are not duplicated in this evidence package.

Baseline and current Rust integration

```
forge test --summary --no-match-path 'test/Audit*.t.sol'
cargo +1.94.1 build --release --locked \
  --manifest-path offchain-rust/Cargo.toml \
  --features ffi-tools --bin usd8-ffi
RUN_INTEGRATION=1 forge test --offline --ffi \
  --match-path test/SettlementIntegration.t.sol -vv
```

New reproductions and targeted fuzzing

```
forge test --match-path test/AuditAdversarial.t.sol -vv
forge test --fuzz-runs 10000 --match-test '^testFuzz_' \
  --no-match-path 'test/AuditMainnetFork.t.sol'
forge test --match-path test/AuditMainnetFork.t.sol \
  --fork-url https://ethereum-rpc.publicnode.com \
  --fork-block-number 26196372 -vv
```

Static analysis, build and known failing gate

```
slither . --filter-paths 'lib/|test/|script/' \
  --exclude-dependencies --json slither.json
forge build --sizes
forge config --json
FOUNDRY_PROFILE=kontrol forge test --summary
```

Expected current result: the three adversarial tests pass by observing the issues, the fork and selected fuzz tests pass, and the kontrol command fails to compile at the removed swap call. Do not interpret a reproduction test pass as remediation.

Evidence package contents

tests/: the two new audit files. source-snapshot/: captured src/, Solidity deployment scripts and foundry.toml. source-manifest.json: file and dependency hashes. build-manifest.json: compiler settings and bytecode hashes. Logs: baseline, integration, adversarial, targeted-fuzz, mainnet-fork, formal-executable, Rust build, Slither and sizes. mainnet-read-checks.json: pinned read-only dependency observations.

working-tree-status.txt and working-tree.patch record the original release boundary. source-comparison.json records that production files still matched the audit snapshot. Existing developer changes were preserved.

13. Provenance, severity and limitations

Snapshot identity

First-party src/ digest (SHA-256 of the canonical sorted manifest records):

`8ce05a0c2c942bd850f5000d4c402aed218e6ea89132450822c97bdd31196e67`

The full SHA-256 for each source file and each Solidity dependency is included in source-manifest.json. This digest is not an Ethereum runtime-code hash and does not attest a deployed release.

Dependency	Git revision
forge-std	c90c02ffa470b46d92ffaab093baeb296300870b
openzeppelin-contracts	cd05883078060e0cd8a7bd36636944570dbe1722
openzeppelin-contracts-upgradeable	7bf4727aacdbfaa0f36cbd664654d0c9e1dc52bf
vault-v2	549b113106e505b83952c7b296622534338d5f92

Severity definitions used here

Critical/High: practical unauthorized loss of substantial principal or similarly severe impact, with reachable prerequisites. Medium: material loss or custody/liveness weakness with meaningful prerequisites or mitigation. Low: constrained, extreme-state, operational or privileged-configuration weakness. Confidence describes the reproduced code behavior; it is separate from likelihood.

Important limits

This is an AI-assisted source review and testing report, not an independent audit-firm certification or formal proof. It did not audit every Rust service, the production enclave, signer custody, deployment wallet, frontend, bridge, booster implementation, USDC/Lido/Chainlink code, Morpho internals or future strategy configuration. It did not test the actual future mainnet deployment addresses or broadcast sequence. No guarantee that all vulnerabilities were identified is made.

The evidence includes a full baseline test run and focused additional checks, not exhaustive symbolic exploration. Test mocks and authorized test signatures deliberately isolate contract behavior; the high-share-loss test does not prove that the same loss series would satisfy the honest production TEE at real historical blocks. The malformed-root test deliberately assumes a trusted-boundary failure.

Result

Three reproducible concerns are documented: one conditional Medium and two Low. No permissionless Critical or High-severity funds-drain was reproduced. The present evidence does not clear the current working tree for a large-TVL mainnet launch. Close the release gates, resolve the recovery design, and review the final frozen build before accepting substantial funds.

Audit date: 15 October 2026, Europe/Berlin. Evidence was collected in the local workspace and an isolated copy. No source fix, mainnet transaction, key access, or external publication was performed.